



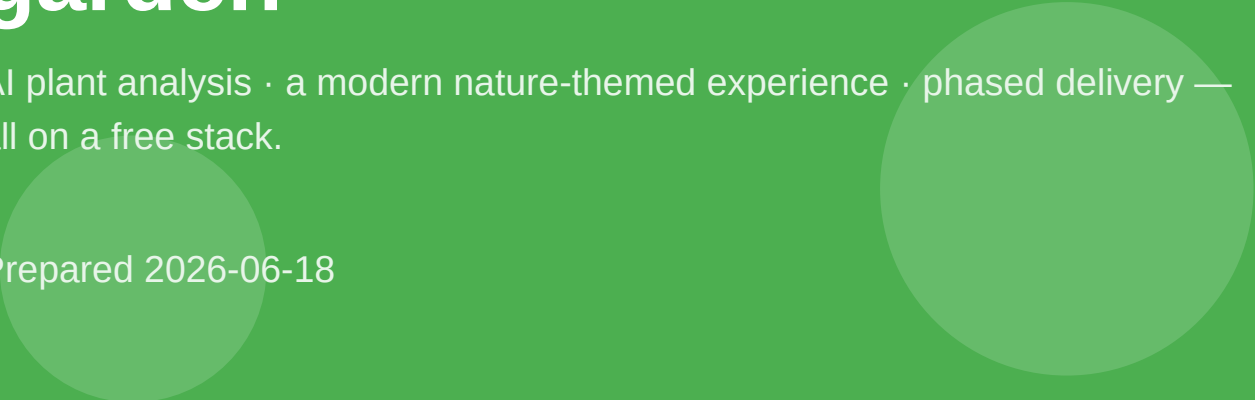
ENHANCEMENT ROADMAP

Growdollar

Growing AI into a living garden

AI plant analysis · a modern nature-themed experience · phased delivery —
all on a free stack.

Prepared 2026-06-18



01 — Vision

Where we are, and where we're growing

Growdollar is already a polished plant-cultivation app: users buy a virtual plant, watch it grow through 11 life stages, water it, photograph its journey, and see its environmental impact. The foundation is strong. This roadmap turns it into something **smart and delightful** — a garden that can look at a real plant and tell you how it's doing, wrapped in a living, animated, nature-themed interface.

The app today

- **11-stage growth simulation** — seed to masterpiece, advancing in real time.
- **Care mechanics** — water, sunlight and care decay and can be restored.
- **Photo journal, location map & environmental impact** (CO₂ / oxygen).
- **Firebase auth** + local AsyncStorage plant data, with Reanimated polish.

Three goals for this phase of work

- **See with AI** — point the camera at a plant; identify the species and diagnose its health.
- **Feel alive** — a modern nature theme: falling leaves, day/night skies, growth celebrations.
- **Feel modern** — tab navigation, onboarding, a real design system and dark mode.

02 — AI Plant Analysis

A garden that can see

The headline feature: a new **AI Scan** tab. The user captures or picks a photo of a plant or leaf; the app identifies the species and reports its health, then offers care tips and lets them attach the insight to one of their plants. Built entirely on a **free stack** — no paid vendor.

The free AI stack

Need	Free option	How it works
Identify species	Pl@ntNet API (<i>free, non-commercial</i>) or iNaturalist vision	Send the photo, get back species name + confidence score.
Diagnose health	Hugging Face Inference (free tier) hosting an open PlantVillage disease model	Classifies healthy vs. common diseases (blight, rust, mildew...).
Care tips	Rule-based mapping (no cost)	Disease label → friendly, actionable advice in the app.
Fully offline option	Bundle a TFLite model via react-native-fast-tflite	No key, no network — slightly larger app, runs on-device.

Flow & data

- Capture / pick photo → loading shimmer → result card (species, confidence, health, diagnosis, tips).
- “Save to plant” attaches the photo and a new **aiInsights** record to a plant in PlantContext.
- Free API key is hidden behind a tiny Firebase Cloud Function proxy — never shipped raw in the bundle.

03 — Modern UX & Nature Animations

A living, catchy interface

Four upgrades make the app feel modern and alive — all using Reanimated, which is already a dependency.

Design system & theming

- Central **theme tokens** (colors, spacing, radius, typography) — replacing today's hardcoded greens.
- **Light / dark mode** with nature palettes: day greens, night deep-teal.

Modern navigation

- Animated **bottom tab bar**: Home · Garden · **AI Scan** · Impact · Profile.
- Floating center “Scan” button; existing detail screens nest as pushed views.

Animated nature theme

- Falling-leaves / floating-pollen particles and gentle plant sway.
- Day/night animated sky gradient tied to device time.
- **Growth-stage-up celebration** — confetti + haptic when a plant levels up.

Onboarding & micro-interactions

- 3–4 screen animated onboarding shown once (growing, AI scan, eco-impact).
- Skeleton loaders, pull-to-refresh, richer haptics, polished empty states.

04 — Phased Roadmap

How we deliver it

Sequenced so each phase stands on its own and de-risks the next. Effort is rough relative sizing.

Phase	Focus	Effort	Value
1	Design system & theming (tokens, dark mode)	Medium	Foundation
2	Modern navigation — animated tab bar	Medium	High
3	AI Plant Scan — headline feature	High	Highest
4	Animated nature theme & celebrations	Medium	High (delight)
5	Onboarding & micro-interactions	Low–Med	Medium
6	Polish, auth-persistence fix, QA & ship	Low	Stability

05 — Tech choices & risks

Things to watch

- **Free-tier limits** — Pl@ntNet / Hugging Face cap requests; cache results and fall back to the on-device TFLite model if rate-limited.
- **Key handling** — keep API keys server-side in a Cloud Function proxy, never in the app bundle.
- **Offline path** — the bundled TFLite model keeps AI scanning working with no network and no key.
- **Auth persistence** — apply the standing Firebase + AsyncStorage fix so login survives restarts.
- **Performance** — keep particle/animation layers opt-in per screen to protect frame rate on older devices.

Next step: begin Phase 1 (design system) — it unblocks every later phase and is low-risk. The AI Scan (Phase 3) is the showcase moment worth demoing first to stakeholders.